

CubeMars AK60-6 V3.0 KV80

Internal notes – V3.0 extended-frame CAN protocol, driver architecture, test scripts

Repo `exo-actuator-sensor-drivers`

May 21, 2026

Contents

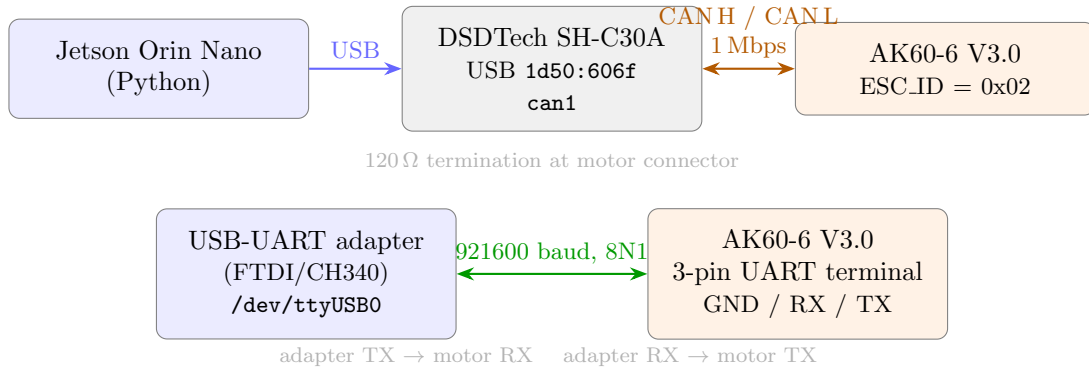
1	Hardware overview	3
1.1	Bench wiring	3
1.2	Adapter setup (once per Jetson OS install)	3
2	V3.0 CAN protocol	3
2.1	Unified extended-ID scheme	3
2.2	R-Link CAN Mode setting	4
3	MIT force-control mode	4
3.1	Torque law	4
3.2	Bit-packing (verbatim from <code>AK-V3.ino</code>)	4
3.3	V3.0 MIT frame byte layout (Kp-first)	5
3.4	Feedback frame (CAN ID 0x2902)	6
4	Servo-mode commands	6
5	Serial UART protocol and parameter read	7
5.1	Frame format (§4.3.2 of the V3.0.1 manual)	7
5.2	<code>COMM_GET_VALUES</code> (0x45) – full status reply	7
6	Python driver architecture	8
6.1	Source file map	8
6.2	<code>AkV3Motor</code> – low-level	8
6.3	<code>AkV3Bench</code> – high-level context manager	8
7	MIT control modes – theory and recommended gains	9
7.1	Position hold	9
7.2	Velocity tracking	9
7.3	Open-loop torque	9
7.4	Recommended gains	9
8	Impedance control and kinesthetic teaching	9
8.1	Back-drivable operation	9
8.2	Recording a demonstration (<code>05_kinesthetic_record.py</code>)	10
8.3	Replaying a demonstration (<code>06_replay_trajectory.py</code>)	10
8.4	Spring-return impedance (<code>07_impedance_spring_return.py</code>)	10

9	Test script inventory	10
9.1	Complete list	10
9.2	Probe and diagnostic scripts (00-*)	10
9.3	01_check_connection.py	11
9.4	02_mit_position.py	11
9.5	03_mit_velocity.py	11
9.6	04_mit_torque.py	11
9.7	05--07	11
10	Calibration (R-Link, one time)	11
11	Known issues and gotchas	12
12	Quick-start checklist	12

1 Hardware overview

Property	Value
Model	AK60-6 V3.0 KV80
Voltage	18–52 V (rated 48 V)
Rated / peak current	10 A / 30 A
Gear ratio	6:1 planetary
Output torque (cont.)	$\approx 5 \text{ N}\cdot\text{m}$
Output torque (peak)	$12 \text{ N}\cdot\text{m}$
Output speed (no-load)	$\approx 60 \text{ rad/s}$ at 48 V
MIT position range	$\pm 12.56 \text{ rad}$ (P_MAX)
Encoder	dual 14-bit magnetic (rotor + output shaft)
Pole pairs	14
CAN bus	1 Mbps, CAN 2.0B extended frames (29-bit IDs)
CAN adapter (bench)	DSDTech SH-C30A (USB ID 1d50:606f), can1
ESC_ID (bench)	0x02 (set via R-Link)
Feedback CAN ID	$(0x29 \ll 8) \mid 0x02 = 0x2902$
UART serial port	3-pin terminal, 921600 baud, 8N1
Configuration tool	R-Link (CubeMars Windows application)

1.1 Bench wiring



The CAN interface is used for all real-time control (MIT and servo modes). The UART serial interface is used only by `00_read_params.py` to read motor operating parameters via `COMM_GET_VALUES`.

1.2 Adapter setup (once per Jetson OS install)

```
# Build and install gs_usb kernel module:
bash install_gs_usb.sh

# Verify adapter; brings can1 up automatically:
bash test_can_adapter_dstech.sh can1

# Manual bring-up (if udev rule has not fired):
sudo ip link set can1 up type can bitrate 1000000
```

2 V3.0 CAN protocol

2.1 Unified extended-ID scheme

All V3.0 control modes use CAN 2.0B extended (29-bit) frames. The arbitration ID encodes both the packet type and the motor's ESC_ID:

$$\text{CAN_ID} = (\text{PACKET_TYPE} \ll 8) \mid \text{ESC_ID} \quad (1)$$

PACKET_TYPE	Constant name	Payload	CAN_ID (ESC=0x02)
0	SET_DUTY	int32: duty $\times 10^5$	0x00000002
1	SET_CURRENT	int32: amps $\times 10^3$	0x00000102
2	SET_CURRENT_BRAKE	int32: amps $\times 10^3$	0x00000202
3	SET_RPM	int32: ERPM (raw)	0x00000302
4	SET_POS	int32: deg $\times 10^4$	0x00000402
5	SET_ORIGIN_HERE	1 byte: 0=volatile, 1=flash	0x00000502
6	SET_POS_SPD	8 bytes: pos+spd+acc	0x00000602
8	SET_MIT	8 bytes bit-packed (§3)	0x00000802
0x29	Feedback (rx only)	8 bytes: pos+spd+I+T+err	0x00002902

Constants defined in `src/ak_v3_common.py`; `src/ak_v3_can.py` calls `_send_eid(packet_type, payload)`.

2.2 R-Link CAN Mode setting

Controls feedback delivery only – not which command types are accepted. MIT and servo commands work in either mode.

Setting	Behaviour
Periodic Feedback	Motor broadcasts feedback at CAN Fdb Rate (default 50 Hz). Recommended for bench use.
Inquiry Feedback	Motor replies with one frame per command received.

3 MIT force-control mode

3.1 Torque law

The on-board controller implements:

$$\tau_{\text{cmd}} = K_p (q_{\text{des}} - q) + K_d (\dot{q}_{\text{des}} - \dot{q}) + \tau_{ff} \quad (2)$$

Sub-modes are selected by which parameters are nonzero:

Sub-mode	K_p	K_d	q_{des}	$\dot{q}_{\text{des}}$	τ_{ff}
Position hold	> 0	> 0	target	0	0
Velocity tracking	0	> 0	0	target	0
Open-loop torque	0	0	0	0	target

Warning: $K_p > 0$, $K_d = 0$ removes all damping – motor oscillates or runs away. Always set $K_d \geq 0.3$ with position commands.

3.2 Bit-packing (verbatim from AK-V3.ino)

Each float is linearly mapped to an unsigned integer:

$$\text{float_to_uint}(x) = \left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} \cdot (2^N - 1) \right\rfloor \quad (3)$$

$$\text{uint_to_float}(u) = \frac{u}{2^N - 1} \cdot (x_{\max} - x_{\min}) + x_{\min} \quad (4)$$

Bit widths and ranges for AK60-6 (from `src/ak_v3_common.py`):

Field	Symbol	Bits (N)	$x_{\min}$	$x_{\max}$
Position	q	16	-12.56 rad	$+12.56 \text{ rad}$
Velocity	$\dot{q}$	12	-60 rad/s	$+60 \text{ rad/s}$
Stiffness	K_p	12	0	500
Damping	K_d	12	0	5
Feed-forward	τ_{ff}	12	$-12 \text{ N}\cdot\text{m}$	$+12 \text{ N}\cdot\text{m}$

```

1 # src/ak_v3_common.py
2 def float_to_uint(x, x_min, x_max, bits):
3     span = x_max - x_min
4     x = max(x_min, min(x_max, x))
5     return int((x - x_min) * ((1 << bits) - 1) / span)
6
7 def uint_to_float(u, x_min, x_max, bits):
8     return u * (x_max - x_min) / ((1 << bits) - 1) + x_min

```

3.3 V3.0 MIT frame byte layout (Kp-first)

V3.0 places K_p in bytes 0–1 (**Kp-first**), unlike the classic mini-cheetah layout (position-first) used by earlier T-Motor firmware:

Byte	Content
0	Kp[11:4]
1	Kp[3:0] Kd[11:8]
2	Kd[7:0]
3	pos[15:8]
4	pos[7:0]
5	vel[11:4]
6	vel[3:0] tau[11:8]
7	tau[7:0]

Sent at extended CAN ID $(8 \ll 8)|0x02 = 0x0802$. Implemented in `AkV3Motor.set_mit()`:

```

1 def set_mit(self, p_des, v_des, kp, kd, t_ff):
2     L = self.limits
3     p_int = float_to_uint(p_des, L.p_min, L.p_max, 16)
4     v_int = float_to_uint(v_des, L.v_min, L.v_max, 12)
5     kp_int = float_to_uint(kp, 0.0, L.kp_max, 12)
6     kd_int = float_to_uint(kd, 0.0, L.kd_max, 12)
7     t_int = float_to_uint(t_ff, L.t_min, L.t_max, 12)
8     buf = bytes([
9         (kp_int >> 4) & 0xFF,
10        ((kp_int & 0xF) << 4) | ((kd_int >> 8) & 0xF),
11        kd_int & 0xFF,
12        (p_int >> 8) & 0xFF,
13        p_int & 0xFF,
14        (v_int >> 4) & 0xFF,
15        ((v_int & 0xF) << 4) | ((t_int >> 8) & 0xF),
16        t_int & 0xFF,
17    ])
18    self._send_eid(CAN_PACKET_SET_MIT, buf)

```

3.4 Feedback frame (CAN ID 0x2902)

Bytes	Encoding	Meaning
0–1	int16 BE, $\times 0.1$	Position (degrees)
2–3	int16 BE, $\times 10$	Speed (ERPM)
4–5	int16 BE, $\times 0.01$	Phase current I_q (A)
6	int8	Driver-board MOS temperature ($^{\circ}\text{C}$)
7	uint8	Error code (0 = no fault)

Decoded in `AkV3Motor.parse_feedback()`:

```

1 def parse_feedback(self, msg):
2     if msg.arbitration_id != (FEEDBACK_PACKET_TYPE << 8) | self.can_id:
3         return False
4     d = msg.data
5     self.pos_deg = int.from_bytes(d[0:2], "big", signed=True) * 0.1
6     self.spd_erpm = int.from_bytes(d[2:4], "big", signed=True) * 10.0
7     self.current_a = int.from_bytes(d[4:6], "big", signed=True) * 0.01
8     self.temp_c = d[6]
9     self.error = d[7]
10    return True

```

`AkV3Bench.read_state()` converts to SI units:

$$q_{\text{rad}} = q_{\text{deg}} \cdot \frac{\pi}{180} \quad (5)$$

$$\dot{q}_{\text{rad/s}} = \frac{\text{ERPM}}{p_{\text{pairs}}} \cdot \frac{2\pi}{60} \quad (p_{\text{pairs}} = 14 \text{ for AK60-6}) \quad (6)$$

4 Servo-mode commands

Servo commands run the firmware’s internal PID loops (no manual gain tuning).

Method on <code>AkV3Motor</code>	<code>write()</code> data name	Notes
<code>set_position_deg(deg)</code>	<code>goal_position_deg</code>	Firmware position PID; deg in $[-36000, 36000]$
<code>set_rpm(erpm)</code>	<code>goal_speed_erpm</code>	Firmware velocity PID
<code>set_pos_spd(deg, erpm, acc)</code>	<code>goal_pos_spd</code>	Trapezoidal position-velocity profile
<code>set_duty(frac)</code>	<code>goal_duty</code>	Open-loop duty in $[-1, 1]$
<code>set_current(amps)</code>	—	Direct I_q current loop
<code>set_brake_current(amps)</code>	—	Braking current hold
<code>set_origin(permanent)</code>	—	Latch current angle as zero

Trapezoidal move example:

```

1 with open_ak_v3_bench() as bus:
2     # Move to 90 deg, cruise at 1000 ERPM, 30000 ERPM/s^2 accel:
3     bus.write("goal_pos_spd", {"j1": 90.0},
4               dq_des=1000, acc_erpm_s2=30_000)

```

5 Serial UART protocol and parameter read

5.1 Frame format (§4.3.2 of the V3.0.1 manual)

Byte(s)	Content
0	Header 0xAA
1	Data length N (command byte + payload)
2	Command byte (<code>comm_id</code>)
3..N+1	Payload bytes
N+2	CRC16 high byte
N+3	CRC16 low byte
N+4	Tail 0xBB

CRC-16 over bytes 2..(N+1) using CCITT polynomial table (§4.3.2.3). Verification: `crc16([0x45])` = 0x1861, giving the full `COMM_GET_VALUES` request: AA 01 45 18 61 BB.

5.2 `COMM_GET_VALUES (0x45)` – full status reply

Data bytes	Type	Field	Scale
0	cmd byte	(0x45)	—
1–2	int16 BE	MOS temperature	$\div 10^\circ\text{C}$
3–4	int16 BE	Motor temperature	$\div 10^\circ\text{C}$
5–8	int32 BE	Output current	$\div 100\text{ A}$
9–12	int32 BE	Input current	$\div 100\text{ A}$
13–16	int32 BE	I_d current	$\div 100\text{ A}$
17–20	int32 BE	I_q current	$\div 100\text{ A}$
21–22	int16 BE	Duty cycle	$\div 1000$
23–26	int32 BE	Motor speed	ERPM
27–28	int16 BE	Input voltage	$\div 10\text{ V}$
29–52	—	Reserved 24 bytes	—
53	uint8	Status/error code	—
54–57	float32 BE	Outer-loop position	rad
58	uint8	Motor ID	—
59–64	—	Reserved 6 bytes	—
65–68	int32 BE	V_d voltage	$\div 1000\text{ V}$
69–72	int32 BE	V_q voltage	$\div 1000\text{ V}$
73–76	int32 BE	Control mode	raw
77–80	float32 BE	Encoder angle	deg
81–84	float32 BE	Outer encoder angle	deg

`COMM_GET_VALUES_SETUP (0x10)` accepts a 4-byte big-endian bitmask to select a subset of fields (returned in the same order).

Usage:

```
sudo python3 tests/cubemars/00_read_params.py [--port /dev/ttyUSB0]
```

6 Python driver architecture

6.1 Source file map

File	Role
src/ak_v3_common.py	Constants: AK_V3_LIMITS, packet-type IDs, float_to_uint, uint_to_float
src/ak_v3_can.py	AkV3Motor – low-level single-motor CAN driver
src/_common.py	AkV3Bench + open_ak_v3_bench() – high-level context-manager API used by all test scripts
src/motor_config.py	CubeMarsMotorConfig, CubeMarsBusConfig, DEFAULT_CUBEMARS_BENCH_CONFIG
tests/cubemars/	Probe/diagnostic scripts (00-*) and control scripts (01-07)

6.2 AkV3Motor – low-level

Wraps a `can.BusABC` (SocketCAN) and exposes one method per packet type, plus `parse_feedback()` / `poll_feedback()`.

```
1 import can
2 from ak_v3_can import AkV3Motor
3
4 bus = can.Bus(channel="can1", interface="socketcan")
5 motor = AkV3Motor(bus, can_id=0x02, model="AK60-6")
6
7 motor.set_mit(p_des=0.0, v_des=0.0, kp=0.0, kd=0.0, t_ff=0.0)
8 if motor.poll_feedback(timeout=0.1):
9     print(motor.pos_deg, motor.spd_erpm, motor.current_a)
10
11 bus.shutdown()
```

6.3 AkV3Bench – high-level context manager

Adds lifecycle management (`connect/disconnect/context manager`), `drain/flush`, and a unified `write()` / `read_state()` API that mirrors DamiaoBus.

Context-manager idiom (used by every test script):

```
1 from _common import open_ak_v3_bench
2
3 with open_ak_v3_bench(set_zero=True) as bus:
4     bus.write("goal_position", {"j1": 1.57}, kp=60, kd=1.5)
5     q, dq, ia = bus.read_state()["j1"]
6     # q : output-shaft position (rad)
7     # dq : output-shaft speed (rad/s)
8     # ia : phase current Iq (A) -- NOT torque in N.m
```

write() data names:

data.name	Command	Key kwargs
goal.position	MIT position hold	kp, kd, dq_des, tau_ff
goal.velocity	MIT velocity track	kd, tau_ff
goal.torque	MIT open-loop torque	—
mit.command	Raw MIT pass-through	kp, kd, dq_des, tau_ff
goal.position_deg	Servo position PID	— (degrees)
goal.pos_spd	Servo pos-vel profile	dq_des (ERPM), acc_erpm_s2
goal.speed_erpm	Servo velocity PID	—
goal.duty	Servo duty cycle	—

Reading state:

```

1 # SI units -- q (rad), dq (rad/s), ia (A):
2 q, dq, ia = bus.read_state()["j1"]
3
4 # Native units -- pos (deg), ERPM, ia (A), temp (C), error code:
5 pos, erpm, ia, tc, err = bus.read_raw()["j1"]

```

`read_state()` sends one zero-torque MIT frame first to elicit a fresh feedback reply (in Inquiry-Feedback mode), then polls `poll_feedback()`.

Converting I_q to torque: $\tau [\text{N}\cdot\text{m}] = I_q [\text{A}] \times K_t [\text{N}\cdot\text{m}/\text{A}]$ where $K_t \approx 0.12 \text{ N}\cdot\text{m}/\text{A}$ for AK60-6 KV80 (from R-Link Motor Identification result).

7 MIT control modes – theory and recommended gains

7.1 Position hold

From Eq. (2) with $\dot{q}_{\text{des}} = 0$: $\tau = K_p(q_{\text{des}} - q) - K_d\dot{q}$. Motor acts as a spring (K_p) with viscous damping (K_d).

```

1 bus.write("goal_position", {"j1": math.radians(30)}, kp=60, kd=1.5)
2 q, dq, ia = bus.read_state()["j1"]

```

7.2 Velocity tracking

$K_p = 0, K_d > 0$: $\tau = K_d(\dot{q}_{\text{des}} - \dot{q})$. No position restoring force.

```

1 bus.write("goal_velocity", {"j1": 1.0}, kd=1.5) # 1 rad/s

```

7.3 Open-loop torque

$K_p = K_d = 0$: $\tau_{\text{cmd}} = \tau_{ff}$. No feedback loop.

```

1 bus.write("goal_torque", {"j1": 0.3}) # 0.3 N.m

```

7.4 Recommended gains

Task	K_p	K_d	Notes
Stiff position hold	80–150	2.0–4.0	Heavy load; tune K_d first
Moderate hold	30–60	1.0–2.0	General-purpose bench
Impedance (teaching)	3–10	0.3–0.6	Back-drivable; recording demos
Velocity tracking	0	1.0–3.0	$K_p = 0$ mandatory
Open-loop torque	0	0	Keep $\leq 1 \text{ N}\cdot\text{m}$ unloaded

8 Impedance control and kinesthetic teaching

8.1 Back-drivable operation

With low K_p and K_d , setting $q_{\text{des}} = q_{\text{meas}}$ each tick keeps the spring force zero:

$$\tau_{\text{spring}} = K_p \underbrace{(q_{\text{des}} - q)}_{=0} = 0$$

Only the damping term $-K_d\dot{q}$ resists fast movements. Lower K_d = softer feel; too low causes jitter from encoder noise.

8.2 Recording a demonstration (05_kinesthetic_record.py)

```
1 KP, KD = 5.0, 0.4 # soft: compliant but not jittery
2 with open_ak_v3_bench(set_zero=True) as bus:
3     while True:
4         q, dq, ia = bus.read_state(timeout=0.02)["j1"]
5         bus.write("goal_position", {"j1": q}, kp=KP, kd=KD)
6         rows.append((time.monotonic(), q, dq, ia))
```

Saves to `demo_trajectory_ak80.csv`: columns `t_s`, `q_rad`, `dq_rad_s`, `current_a` at 100 Hz.

8.3 Replaying a demonstration (06_replay_trajectory.py)

Use higher gains during replay for tracking under load:

```
1 bus.write("goal_position", {"j1": q_des[i]}, kp=30, kd=1.0)
```

8.4 Spring-return impedance (07_impedance_spring_return.py)

Fixed anchor at $q = 0$; motor springs back when released:

```
1 # 30-second loop at 100 Hz
2 bus.write("goal_position", {"j1": 0.0}, kp=10, kd=0.5)
```

9 Test script inventory

9.1 Complete list

Script	Purpose
00_loopback_dsd_tech.py	DSDTech adapter loopback (software + physical)
00_loopback_socketcan.py	Jetson CAN0 loopback (software + physical)
00_scan_bitrate.py	Passive bitrate scan; confirm 1 Mbps
00_probe_motor.py	Zero-torque CAN ping; verify ESC_ID and latency
00_sniff_motor.py	Passive sniffer; decode feedback frames, no TX
00_read_params.py	UART serial parameter read (COMM_GET_VALUES)
01_check_connection.py	Poll 5 feedback frames; confirm non-zero
02_mit_position.py	MIT position hold; step through target angles
03_mit_velocity.py	MIT velocity tracking
04_mit_torque.py	Open-loop torque pulses (≤ 1 N·m)
05_kinesthetic_record.py	Hand-guide + save <code>demo_trajectory_ak80.csv</code>
06_replay_trajectory.py	Replay recorded CSV
07_impedance_spring_return.py	Soft virtual spring anchored at zero

All scripts use `with open_ak_v3_bench(...) as bus:` and run as `sudo python3`.

9.2 Probe and diagnostic scripts (00_*)

`00_loopback_dsd_tech.py` Software loopback (echo without motor) then physical loopback (CAN H shorted to CAN L). Run first when the DSDTech adapter is new or suspected faulty.

`00_scan_bitrate.py` Passively listens on `can1` with no TX. Confirms the motor broadcasts at 1 Mbps and shows the feedback CAN ID (0x2902).

`00_probe_motor.py` Sends one zero-torque MIT frame and waits for the feedback reply. First-contact test before any control script.

`00_sniff_motor.py` Listen-only mode; decodes and prints every feedback frame without transmitting. Safe to run while the motor is powered and moving. Verifies R-Link CAN Mode and ESC_ID.

`00_read_params.py` Requires a separate USB-UART adapter on the motor's 3-pin UART terminal. Sends `COMM_GET_VALUES` (0x45) and parses the 73-byte reply; demonstrates `COMM_GET_VALUES_SETUP` bitmask query.

9.3 01_check_connection.py

Polls five feedback frames and prints native units:

```
1 with open_ak_v3_bench() as bus:
2     for i in range(5):
3         pos, erpm, ia, tc, err = bus.read_raw(timeout=0.2)["j1"]
4         print(f"pos={pos:+8.2f} deg spd={erpm:+8.1f} ERPM "
5               f"I={ia:+6.2f} A T={tc:3d} C err={err}")
6         time.sleep(0.2)
```

9.4 02_mit_position.py

Steps through `TARGETS_DEG` = [0.0, 15.0, -15.0, 30.0, 0.0], holding each for 1.5 s at $K_p=60$, $K_d=1.5$:

```
1 with open_ak_v3_bench(set_zero=True) as bus:
2     for tgt_deg in TARGETS_DEG:
3         tgt_rad = math.radians(tgt_deg)
4         t0 = time.monotonic()
5         while time.monotonic() - t0 < HOLD_S:
6             bus.write("goal_position", {"j1": tgt_rad}, kp=60, kd=1.5)
7             q, dq, ia = bus.read_state(timeout=0.02)["j1"]
```

9.5 03_mit_velocity.py

Commands velocity targets (rad/s) via MIT velocity mode ($K_p = 0$, $K_d > 0$). Prints actual vs target speed at 5 Hz.

9.6 04_mit_torque.py

Short open-loop torque pulses ($\tau_{ff} \leq 1 \text{ N}\cdot\text{m}$):

```
1 bus.write("goal_torque", {"j1": 0.5}) # apply 0.5 N.m
2 time.sleep(0.5)
3 bus.write("goal_torque", {"j1": 0.0}) # release
4 time.sleep(0.5)
```

9.7 05--07

See Section 8 for detailed descriptions and code.

10 Calibration (R-Link, one time)

1. **Motor Identification** – R-Link spins the motor to measure resistance, inductance, and back-EMF. Motor must be free to spin.

2. **Encoder Identification** – samples encoder at many rotor positions; builds correction table. Run immediately after Motor ID.
3. **Write Parameters** – click *Write*. Save `.AppParams` and `.McParams` as backups.
4. **CAN settings** – set CAN Bitrate = 1000 kbps, CAN ID = 2, CAN Mode = Periodic Feedback.

Re-calibrate after swapping motor phase wires, replacing the motor, or flashing new firmware.

Software zero (volatile):

```
1 with open_ak_v3_bench(set_zero=True) as bus: ...
2 # or explicitly after connect:
3 bus.set_zero() # permanent=False by default (RAM only, resets on power)
```

11 Known issues and gotchas

1. **Never $K_d = 0$ in position mode.** $K_p > 0$, $K_d = 0$ removes all damping; motor oscillates or runs away. Always set $K_d \geq 0.3$ with position commands.
2. **ESC_ID = 0x02.** Wrong ESC_ID causes `read_state()` to return cached zeros silently. Verify with `00_probe_motor.py`; change via R-Link.
3. **can1 missing after reboot.** Replug the DSDTech adapter (udev rule brings `can1` up), or:

```
sudo ip link set can1 up type can bitrate 1000000
```

4. **gs_usb “Unexpected unused echo id” in dmesg.**

```
sudo rmmod gs_usb && sudo modprobe gs_usb
```

5. **`read_state()` tau field is amps, not N·m.** Third element `ia` of `(q, dq, ia)` is I_q in amps. Multiply by $K_t \approx 0.12 \text{ N·m/A}$ for torque.
6. **V3.0 ignores 11-bit standard CAN frames.** The legacy `cubemars_bus.py` silently fails on V3.0 firmware. Use `ak.v3_can.py` / `AkV3Bench` exclusively.
7. **Calibration required before first use.** An uncalibrated motor may spin erratically. Run R-Link Motor ID + Encoder ID (Section 10).
8. **Peak current caution.** Motor allows up to 30 A peak. Keep $\tau_{ff} \leq 1 \text{ N·m}$ when unloaded (`04_mit_torque.py`).
9. **`00_read_params.py` requires a USB-UART adapter.** The DSDTech SH-C30A is a CAN-only adapter. Use a separate FTDI/CH340 dongle wired to the motor’s 3-pin UART terminal.

12 Quick-start checklist

1. Power on motor (18–52 V).
2. Plug DSDTech SH-C30A into Jetson USB. Confirm:

```
ip -details link show can1
```

3. Passive sniff – confirm motor broadcasting:

```
sudo python3 tests/cubemars/00_sniff_motor.py
```

Expected: frames with CAN ID 0x2902 at 50 Hz.

4. Probe motor – verify ESC_ID and round-trip:

```
sudo python3 tests/cubemars/00_probe_motor.py
```

5. Connection check – 5 frames, non-zero values:

```
sudo python3 tests/cubemars/01_check_connection.py
```

6. MIT position sweep:

```
sudo python3 tests/cubemars/02_mit_position.py
```

7. Proceed with scripts 03–07 as needed.